

# Octopus CXL Memory Pooling: Trace Data Exploration

Pulak Mehrotra

March 20, 2026

## Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1 Overview</b>                                         | <b>2</b>  |
| <b>2 Data Loading and VM Schema</b>                       | <b>2</b>  |
| 2.1 VM Class Fields . . . . .                             | 2         |
| 2.2 Machine and VM Type Sizes . . . . .                   | 2         |
| <b>3 Trace Inventory</b>                                  | <b>2</b>  |
| <b>4 The Environment and Simulation Model</b>             | <b>3</b>  |
| 4.1 Key Concepts . . . . .                                | 3         |
| 4.2 The HOTFIX Filter . . . . .                           | 3         |
| 4.3 Reward . . . . .                                      | 3         |
| <b>5 rss[1] Distribution (Fig 1)</b>                      | <b>4</b>  |
| <b>6 VM Lifetime Distribution (Fig 2)</b>                 | <b>4</b>  |
| 6.1 Data Quality Summary Across Traces . . . . .          | 5         |
| <b>7 Per-Node Statistics (Fig 3)</b>                      | <b>5</b>  |
| <b>8 VM Arrival Rate (Figs 4 and B)</b>                   | <b>6</b>  |
| <b>9 Negative Lifetime Deep-Dive (Figs D)</b>             | <b>7</b>  |
| <b>10 HOTFIX Survival vs. Scale Factor (Fig 5)</b>        | <b>8</b>  |
| <b>11 Episode Event Count Distribution (Figs E and F)</b> | <b>9</b>  |
| <b>12 Other rss Fields (Fig 6)</b>                        | <b>10</b> |
| <b>13 Data Augmentation Design</b>                        | <b>10</b> |
| 13.1 Why Episode-Level Augmentation . . . . .             | 11        |
| 13.2 Augmentation Knobs . . . . .                         | 11        |
| 13.3 Validity Guards . . . . .                            | 11        |
| 13.4 Recommended Sampling Policy . . . . .                | 11        |
| 13.5 Expected Effect on RL Training . . . . .             | 12        |
| 13.6 Implementation Sketch (Pseudocode) . . . . .         | 12        |

# 1 Overview

This document records the data exploration and diagnostic experiments conducted on the Azure VM traces used to train the Octopus CXL memory pooling RL agent. The goal is to understand the structure of the raw data before designing a data augmentation pipeline.

All experiments use ten production Azure VM traces collected over an approximately 8-day window in November–December 2023, stored as pre-processed pickle files in `data/traces/`. Each pickle unpacks to a tuple (`all_vms`, `node_to_vms`, `node_to_machine`, `vm_type_sz`, `machine_sz`) where each VM is an instance of the VM class defined in `octopus/data.py`.

## 2 Data Loading and VM Schema

### 2.1 VM Class Fields

Each VM record contains the following fields relevant to the simulation:

| Field                   | Type                      | Description                                                    |
|-------------------------|---------------------------|----------------------------------------------------------------|
| <code>rss[1]</code>     | float (MB)                | Memory allocation size — <b>the only field read by the env</b> |
| <code>start_time</code> | datetime                  | VM arrival time                                                |
| <code>end_time</code>   | datetime                  | VM termination time                                            |
| <code>max_mem_ts</code> | list of [datetime, float] | Per-tick memory utilisation (%)                                |
| <code>avg_mem_ts</code> | list of [datetime, float] | Per-tick average memory utilisation (%)                        |
| <code>rss[0]</code>     | float                     | CPU-related resource (not used by env)                         |
| <code>rss[2]</code>     | float                     | Storage resource (not used by env)                             |
| <code>rss[3]</code>     | float                     | Network/IOPS resource (not used by env)                        |

The environment (`octopus/env.py`) reads exactly one field per VM: `rss[1]`, the static memory allocation size. The time series fields `max_mem_ts` and `avg_mem_ts` are never consumed. Values in those time series are utilisation percentages (observed values of 43–44 against an `rss[1]` of 7168 MB), so actual consumed memory at tick  $t$  would be  $\text{rss}[1] \times \text{avg\_mem\_ts}[t]/100$ . This richer signal is ignored in the current implementation.

### 2.2 Machine and VM Type Sizes

`machine_sz` maps machine type  $\rightarrow$  a list of four resource capacities indexed identically to `rss`. Index 1 is physical DRAM. In the AMS20 trace there are four machine types, all with identical capacity vectors `[40000, 458812, 5494272, 40000000]`, meaning all nodes have 458 GB of physical DRAM. `vm_type_sz` has a similar layout for VM SKUs.

## 3 Trace Inventory

Ten traces are available, all spanning approximately the same 8-day window (2023-11-20 to 2023-11-28):

| Trace                            | Location   | Total VMs |
|----------------------------------|------------|-----------|
| AMS20PrdApp19-tround.sqlite      | Amsterdam  | 34,029    |
| BLAPrdApp19-troundgrt5m.sqlite   | Bangalore  | 129,698   |
| BN9PrdApp18-troundgrt5m.sqlite   | BN9        | 178,224   |
| DSM08PrdApp05-troundgrt5m.sqlite | Des Moines | 103,847   |
| DUB24PrdApp09-troundgrt5m.sqlite | Dublin     | 79,151    |
| LON23PrdApp01-troundgrt5m.sqlite | London     | 23,450    |
| LVL01PrdApp05-troundgrt5m.sqlite | Louisville | 271,080   |
| SG2PrdApp35-troundgrt5m.sqlite   | Singapore  | 141,018   |
| SYD21PrdApp07-troundgrt5m.sqlite | Sydney     | 57,501    |
| YTO21PrdApp05-troundgrt5m.sqlite | Toronto    | 80,280    |

## 4 The Environment and Simulation Model

### 4.1 Key Concepts

**Node.** A physical server in the datacenter. Each node hosts some set of VMs and has a physical DRAM capacity from `machine.sz`.

**Pod.** A randomly selected subset of `pod.size` nodes (default: 16) that share CXL memory for one training episode. Pods are re-sampled every episode by shuffling the node list with a seed derived from the episode count.

**Event.** One VM arrival that requires an agent allocation decision. Each event is a tuple (`arrival_tick`, `host_id`, `rss[1]`, `dealloc_tick`). The number of events per episode equals the number of VMs on the selected pod nodes that survive the HOTFIX filter.

### 4.2 The HOTFIX Filter

`_build_events` in `env.py` runs a greedy sequential filter over each node before building the event list. VMs are processed in arrival order; if adding a VM would push the node’s running memory total above `machine.sz[node_type][1]`, the VM is added to `vmkey_to_skip` and excluded from the episode. This enforces physical DRAM feasibility. VMs with `vm_e < 0` (end tick before pod window start) are also skipped.

### 4.3 Reward

At each VM arrival the reward is:

$$r = -\frac{\Delta \max_j c_j}{\text{fair\_share}} - \lambda \cdot \text{Var}\left(\frac{c}{\text{fair\_share}}\right)$$

where  $c_j$  is the current load on MPD  $j$ , `fair_share` = `pod_dram/num_mhd`, and  $\lambda = 0.5$  is the variance penalty weight.

## 5 `rss[1]` Distribution (Fig 1)

Figure 1: `rss[1]` (env memory signal) distribution

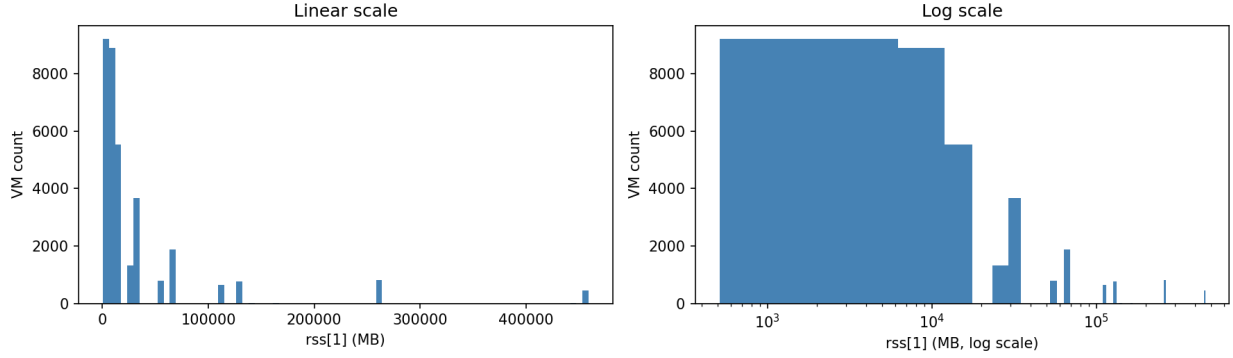


Figure 1: Distribution of `rss[1]` across all VMs in the AMS20 trace. Left: linear scale. Right: log scale. Summary statistics: min=512 MB, p5=2048 MB, p50=8192 MB, p95=131072 MB, max=458812 MB.

`rss[1]` is strongly discrete, clustering at powers of two (512, 1024, 2048, 4096, 8192 MB, ...) because Azure VMs are provisioned in standard SKU sizes. This has a direct implication for augmentation: per-VM scaling would destroy the SKU boundaries, creating memory sizes that do not exist in production. **Augmentation must therefore operate at the episode level** (a single scale factor applied uniformly to all VMs in one episode), not per-VM.

## 6 VM Lifetime Distribution (Fig 2)

Figure 2: VM lifetime distribution

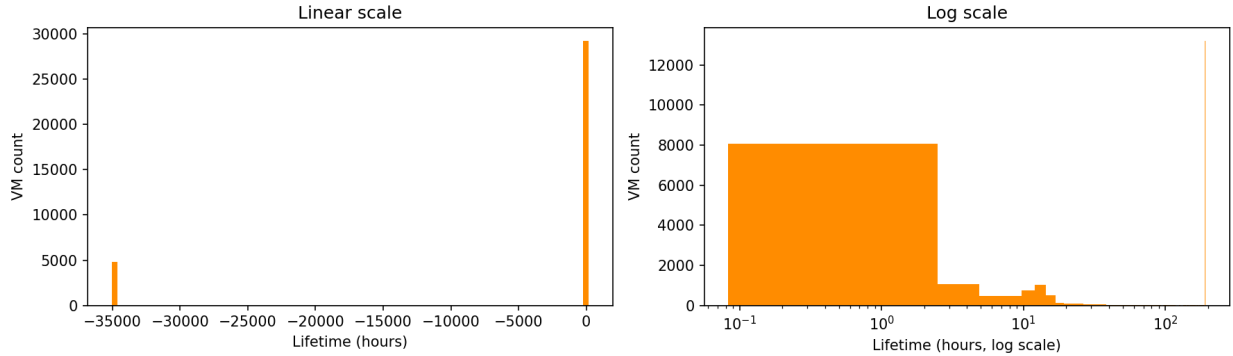


Figure 2: VM lifetime distribution for AMS20. Left: linear scale (negative lifetimes visible). Right: log scale of positive lifetimes only.

Approximately 14% of VMs in AMS20 have `end_time` < `start_time`, yielding negative computed lifetimes. Fig D confirms these all share a single negative depth of  $\sim 35,000$  hours, corresponding to the default unset value `end_time = datetime(2020, 1, 1, 1, 1)` combined with a `start_time`

in November 2023. A threshold filter of `end_time < datetime(2021, 1, 1)` cleanly identifies all such VMs. Similarly,  $\sim 6\%$  of VMs have zero lifetimes (`start_time == end_time`).

All negative-lifetime VMs are caught by the HOTFIX filter (potentially NOT caught by HOTFIX: 0 across all ten traces), so they do not corrupt the simulation. They must however be excluded when computing statistics for augmentation parameter estimation.

## 6.1 Data Quality Summary Across Traces

| Trace | Total VMs | Neg. lifetime | Zero lifetime | Pos. lifetime | Spike VMs |
|-------|-----------|---------------|---------------|---------------|-----------|
| AMS20 | 34,029    | 14.1%         | 6.1%          | 79.8%         | 17.1%     |
| BLA   | 129,698   | 16.5%         | 6.1%          | 77.4%         | 11.4%     |
| BN9   | 178,224   | 15.5%         | 6.9%          | 77.6%         | 7.8%      |
| DSM   | 103,847   | 12.1%         | 7.7%          | 80.2%         | 10.4%     |
| DUB   | 79,151    | 10.6%         | 8.0%          | 81.5%         | 14.3%     |
| LON   | 23,450    | 15.1%         | 8.9%          | 76.0%         | 8.8%      |
| LVL   | 271,080   | 18.7%         | 14.3%         | 67.0%         | 3.4%      |
| SG2   | 141,018   | 16.9%         | 5.4%          | 77.6%         | 10.9%     |
| SYD   | 57,501    | 14.8%         | 4.3%          | 80.9%         | 11.8%     |
| YTO   | 80,280    | 19.5%         | 8.0%          | 72.5%         | 10.0%     |

“Spike VMs” are those with `start_time == trace_min_start`, i.e. VMs already running when trace collection began, whose arrival time was set to the first tick of the trace window.

## 7 Per-Node Statistics (Fig 3)

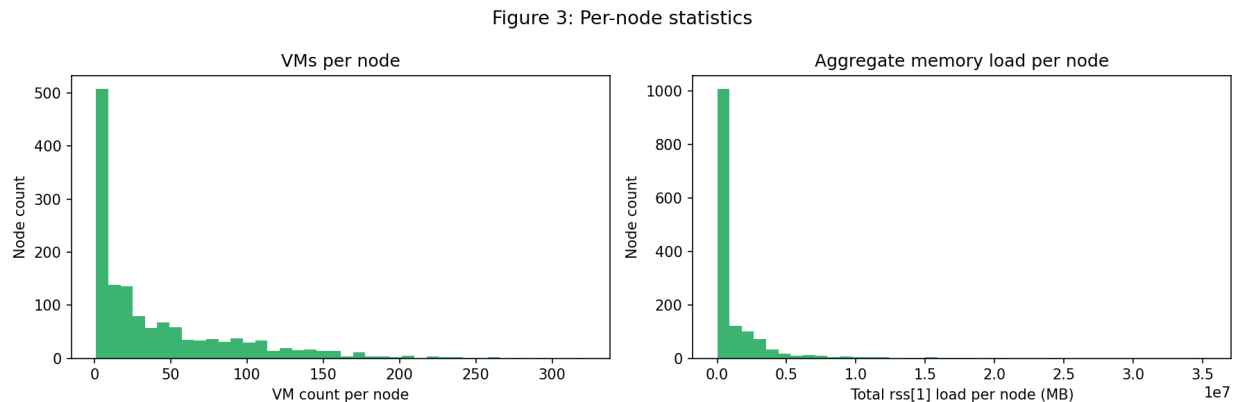


Figure 3: Per-node statistics for AMS20. Left: VM count per node. Right: aggregate `rss[1]` load per node.

Both distributions are right-skewed. The majority of nodes host fewer than 20 VMs and carry modest aggregate load, while a small number of nodes are substantially heavier. This heterogeneity means that pod episode density varies considerably depending on which nodes are sampled.

## 8 VM Arrival Rate (Figs 4 and B)

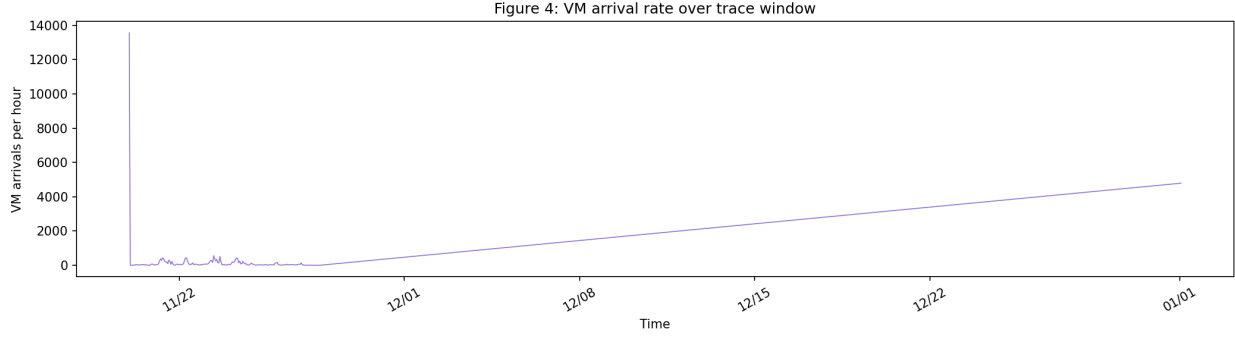


Figure 4: Raw hourly VM arrival rate for AMS20. The large spike at trace start and apparent linear ramp are both artifacts (see text).

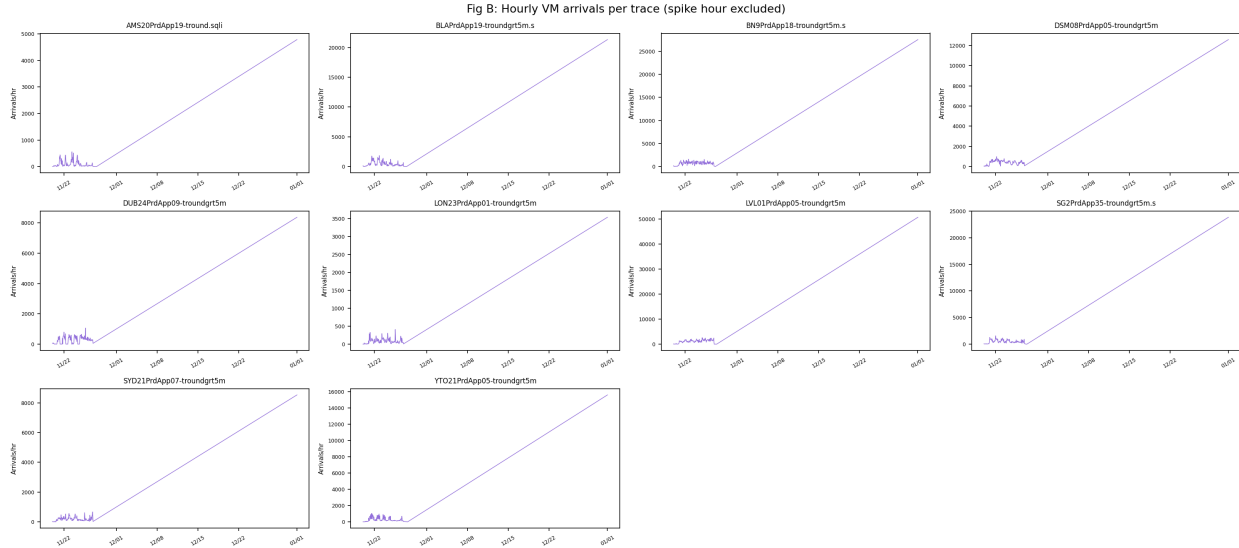


Figure 5: Hourly VM arrivals per trace with the spike hour excluded. The linear ramp visible in Fig 4 is a matplotlib rendering artifact caused by a cluster of VMs with default `start_time = datetime(2024, 1, 1, 1, 1)` binning to a distant future hour.

Two artifacts are present in the raw arrival rate plot:

1. **Trace-start spike.** VMs already running at collection time are assigned `start_time = trace_min_start`, creating a large artificial burst at the first tick (7–17% of all VMs depending on trace). Fig C confirms these spike VMs have the same `rss[1]` distribution as normal arrivals, so they are not a biased subsample.
2. **Linear ramp.** VMs with the default `start_time = datetime(2024, 1, 1, 1, 1)` bin to a single future hour far outside the trace window. matplotlib draws a straight line from the last real data point to that distant bin, producing the apparent ramp. The actual arrival signal (diurnal, with appropriate variation) is visible only in the left portion of Fig 4 and clearly in

Fig B after the spike hour is removed.

## 9 Negative Lifetime Deep-Dive (Figs D)

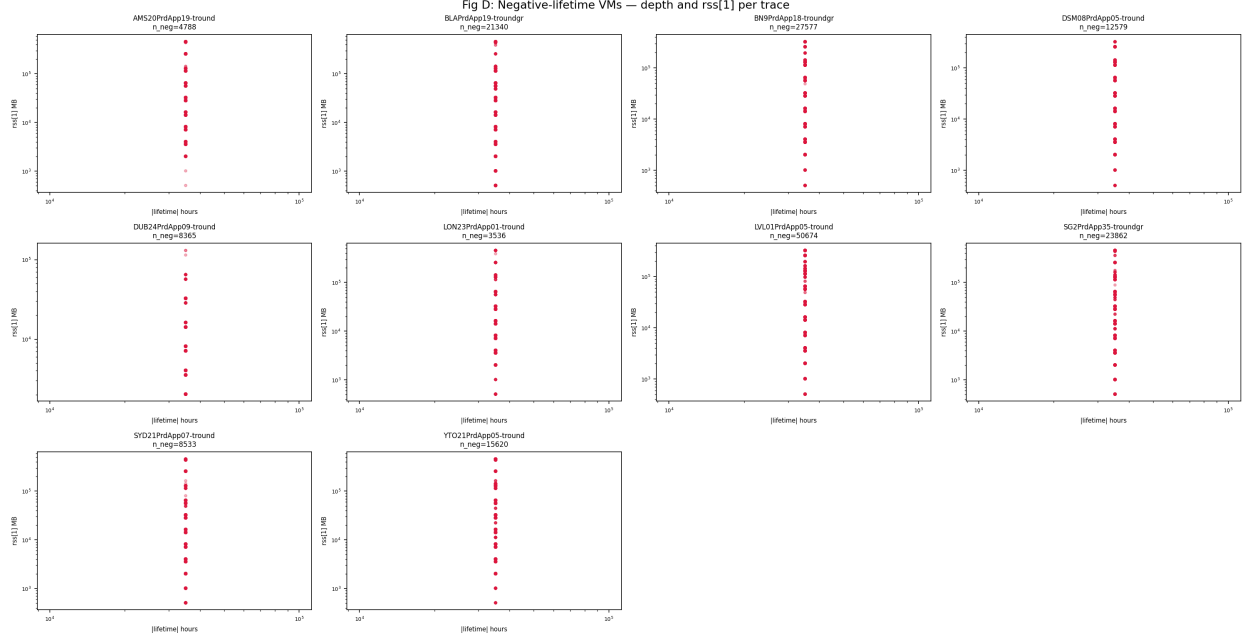


Figure 6: Scatter of  $|\text{lifetime}|$  vs `rss[1]` for negative-lifetime VMs per trace. All points cluster at a single x-value ( $\sim 35,000$ – $40,000$  hours), confirming a single unset default rather than varied corruption.

All negative-lifetime VMs share the same negative depth regardless of their `rss[1]`. The depth corresponds exactly to `datetime(2020,1,1)` subtracted from a November 2023 `start_time`. A single threshold suffices to identify them: `end_time < datetime(2021, 1, 1)`.

## 10 HOTFIX Survival vs. Scale Factor (Fig 5)

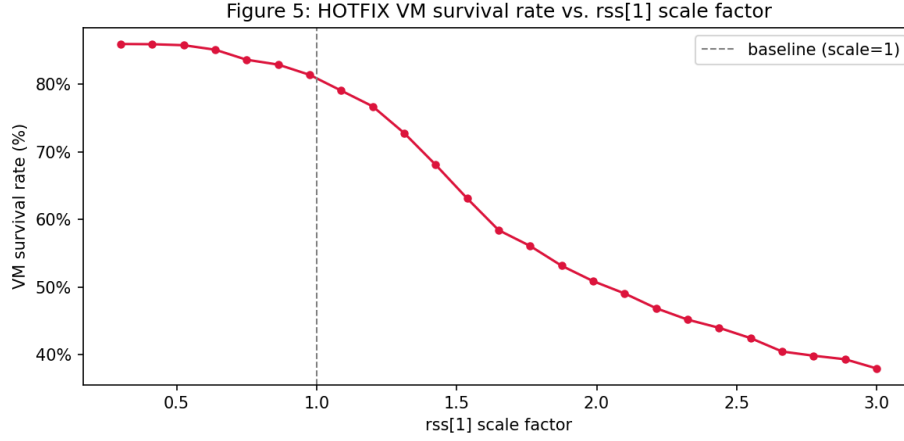


Figure 7: Fraction of VMs surviving the HOTFIX filter as a function of episode-level `rss[1]` scale factor, computed across all nodes in AMS20.

At baseline (scale = 1.0), 81% of VMs survive HOTFIX. The survival curve is approximately flat below scale  $\approx 0.7$  (floor of  $\sim 86\%$  imposed by VMs filtered for reasons other than DRAM overflow) and drops steeply above scale = 1.2:

| Scale factor | Survival rate |
|--------------|---------------|
| 0.5          | 85.8%         |
| 0.7          | 83.6%         |
| 1.0          | 81.0%         |
| 1.2          | 76.7%         |
| 1.5          | 64.9%         |
| 2.0          | 50.7%         |

**Design constraint:** episode-level scaling must remain in the range  $[0.7, 1.2]$  to avoid excessive VM dropout that would produce unrepresentatively sparse training episodes.



## 11 Episode Event Count Distribution (Figs E and F)

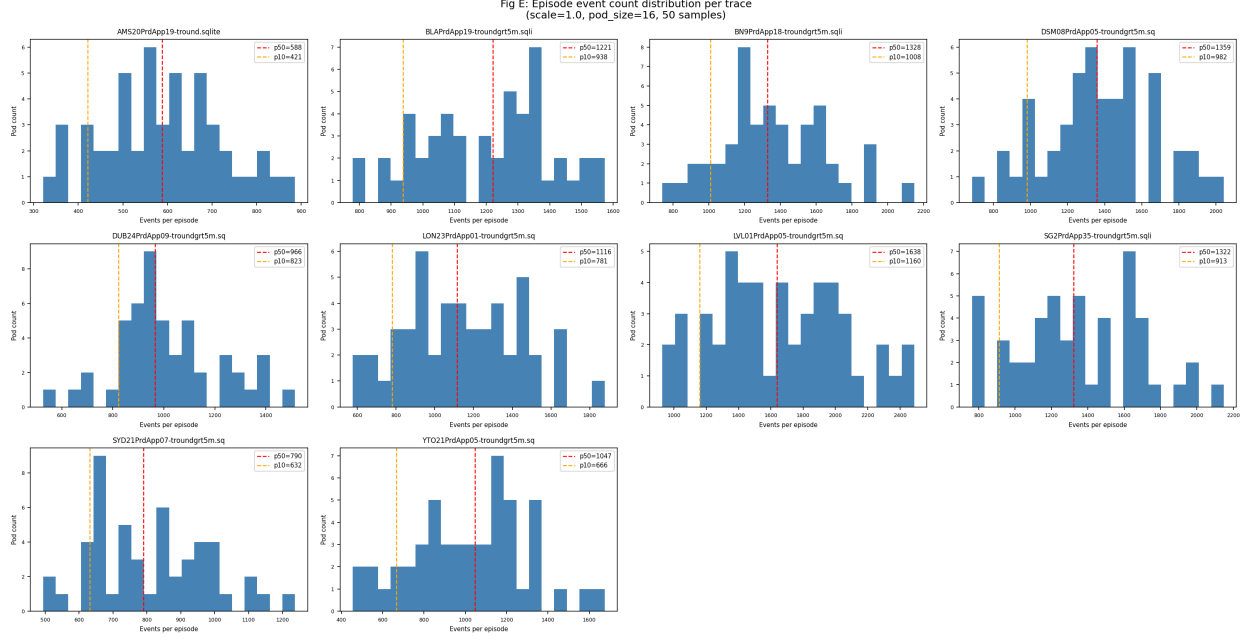


Figure 8: Distribution of events per episode across 50 random pod assignments per trace (scale=1.0, pod\_size=16). Red dashed: p50. Orange dashed: p10.

Episodes are dense. Across all ten traces, p10 ranges from  $\sim 400$  to  $\sim 1000$  events per episode, and p50 ranges from  $\sim 590$  to  $\sim 1640$ . No trace produces near-empty episodes at baseline, confirming that the bad-VM filtering (negative/zero lifetimes) does not materially reduce episode signal.

| Trace | p50 events | p10 events |
|-------|------------|------------|
| AMS20 | 588        | 421        |
| BLA   | 1221       | 938        |
| BN9   | 1328       | 1008       |
| DSM   | 1359       | 982        |
| DUB   | 966        | 823        |
| LON   | 1116       | 781        |
| LVL   | 1638       | 1160       |
| SG2   | 1322       | 913        |
| SYD   | 790        | 632        |
| YTO   | 1047       | 666        |

Given this density, the MIN\_EVENTS guard for augmentation can be set conservatively at 100 events per episode; in practice it will almost never trigger within the safe scaling range of  $[0.7, 1.2]$ .

## 12 Other rss Fields (Fig 6)

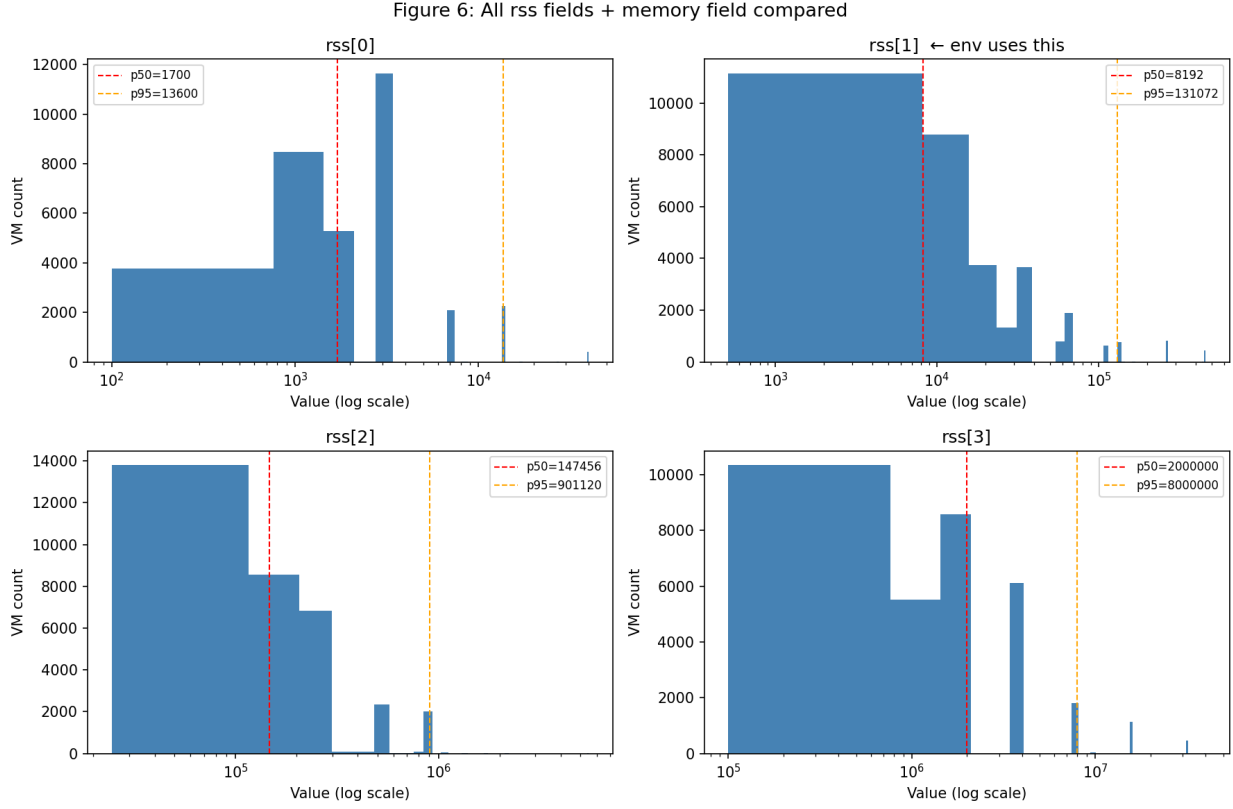


Figure 9: Distributions of all four `rss` fields (log scale) for AMS20. `rss[1]` is the only field used by the env.

| Field               | p50       | p95       | Likely interpretation     |
|---------------------|-----------|-----------|---------------------------|
| <code>rss[0]</code> | 1,700     | 13,600    | vCPU allocation (scaled)  |
| <code>rss[1]</code> | 8,192     | 131,072   | Memory (MB) — used by env |
| <code>rss[2]</code> | 147,456   | 901,120   | Storage (MB)              |
| <code>rss[3]</code> | 2,000,000 | 8,000,000 | Network bandwidth or IOPS |

None of the other fields are a viable substitute for `rss[1]` as a memory demand signal. The `avg_mem_ts` time series (utilisation percentages against `rss[1]`) would provide a richer dynamic demand signal but requires env changes to consume.

## 13 Data Augmentation Design

The augmentation strategy should follow the same principle used in high-throughput physics simulation systems: generate new training episodes by randomizing environment conditions, rather than perturbing individual samples in unrealistic ways. In this project, the correct unit of randomization is the **episode**, not the VM.

### 13.1 Why Episode-Level Augmentation

Fig 1 shows that `rss[1]` is a discrete SKU-like variable (powers-of-two clusters). Per-VM scaling would create synthetic memory sizes that do not exist in production and would break the empirical VM type structure. Therefore, each augmented episode applies a single global factor to all VM memory requests:

$$\text{rss}[1]' = s \cdot \text{rss}[1], \quad s \sim \mathcal{U}(0.7, 1.2).$$

The safe range  $[0.7, 1.2]$  is dictated by HOTFIX survival (Fig 5): lower values provide little additional variation, while higher values cause severe VM dropout and produce unrealistically sparse episodes.

### 13.2 Augmentation Knobs

For each training episode, sample an augmentation configuration:

1. **Trace ID** from the 10-trace corpus (uniform or weighted).
2. **Pod seed** (node subset sampling), already supported by current reset logic.
3. **Memory scale**  $s \in [0.7, 1.2]$  applied uniformly to all VM `rss[1]` in that episode.
4. **Optional arrival jitter** (small bounded tick shifts) to decorrelate exact co-arrivals while preserving chronology.
5. **Optional lifetime perturbation** for valid positive-lifetime VMs only (never applied to known invalid default timestamps).
6. **Optional topology perturbation** via low-rate random edge removal (failure injection), e.g. 1–5% link failures per episode.

This is analogous to domain randomization in simulator-based RL: each episode represents a different, but still plausible, operating regime.

### 13.3 Validity Guards

Each augmented episode must satisfy:

- **HOTFIX feasibility**: apply the same HOTFIX filter after augmentation.
- **Minimum density**: enforce `MIN_EVENTS` = 100.
- **Data quality**: exclude negative-lifetime default artifacts from any augmentation parameter estimation.
- **Topology safety**: reject samples that disconnect hosts from all MPDs; cap failure injection to the graceful-degradation regime (recommended  $\leq 5\%$ ).

If a sampled configuration fails the guards, resample augmentation parameters for that episode.

### 13.4 Recommended Sampling Policy

To avoid over-concentrating around the baseline:

- Sample  $s$  from a clipped triangular distribution centered at 1.0 with support  $[0.7, 1.2]$  for stable yet diverse coverage.

- Use mixed-trace batches so each training window sees both low-utilization and high-utilization datacenter regimes.
- Stratify pod seeds to avoid repeatedly sampling the same easy pods.

### 13.5 Expected Effect on RL Training

Episode-level augmentation increases the diversity of load trajectories without violating SKU structure, improves robustness to cluster-to-cluster and time-to-time demand shifts, and reduces overfitting to a single pod/trace configuration. Most importantly, it preserves simulator mechanics (HOTFIX, event ordering, topology constraints), so policy improvements transfer to non-augmented evaluation runs.

### 13.6 Implementation Sketch (Pseudocode)

```
def sample_episode_config(rng):
    trace_id = sample_trace(rng)                # one of 10 traces
    pod_seed = rng.randint(0, 2**31 - 1)        # controls pod node mapping
    scale = sample_triangular(rng, lo=0.7, mode=1.0, hi=1.2)
    arrival_jitter = rng.choice([0, 0, 1])      # optional, small
    lifetime_noise = rng.choice([0.0, 0.0, 0.05]) # optional, mild
    fail_ratio = rng.choice([0.0, 0.0, 0.01, 0.02, 0.03, 0.05]) # optional
    return {
        "trace_id": trace_id,
        "pod_seed": pod_seed,
        "scale": scale,
        "arrival_jitter": arrival_jitter,
        "lifetime_noise": lifetime_noise,
        "fail_ratio": fail_ratio,
    }

def build_augmented_episode(cfg):
    events = load_events(cfg["trace_id"], pod_seed=cfg["pod_seed"])
    topo = inject_link_failures(base_topology, ratio=cfg["fail_ratio"])
    if not host_connectivity_ok(topo):           # at least 1 MPD/host
        return None
    events = scale_memory(events, factor=cfg["scale"]) # rss[1] only
    events = jitter_arrivals(events, max_shift=cfg["arrival_jitter"])
    events = perturb_lifetimes(events, frac=cfg["lifetime_noise"],
                               positive_only=True)
    events = apply_hotfix(events)                # required
    if len(events) < 100:                        # MIN_EVENTS guard
        return None
    return events
```

In practice, training loop integration is:

1. sample config at `env.reset()`,
2. construct episode events with the augmented pipeline,

3. resample if guards fail,
4. run rollout and log config metadata with rewards/savings.